



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по практическим работам №11-14

по дисциплине

«Системная и программная инженерия»

Выполнили:

Студенты группы ИКБО-66-23

Ковалев А.Э.

Гордильо А.

Смирнов А.Ю.

Тименко И.А.

Проверил:

Левандровский А.М.

Москва 2026

Оглавление

1. ПОДГОТОВКА К ТЕСТИРОВАНИЮ ПРОДУКТА.....	3
1.1. Тестовый случай или тестовый сценарий.....	3
1.2. Инструменты для проведения тестирования.....	7
1.3. Тестирование (не-)функциональных требований.....	7
2. ФОРМИРОВАНИЕ ПРОГРАММЫ И МЕТОДИКИ ИСПЫТАНИЙ СИСТЕМЫ.....	11
2.1. Программа и методика испытаний.....	11
3. АНАЛИЗ И ОЦЕНКА РИСКОВ ПРИ РАЗРАБОТКЕ ПРОГРАММНОГО ПРОЕКТА.....	14
4. АНАЛИЗ ЭКСПЛУАТАЦИИ ПРОДУКТА.....	19
4.1. Технологические угрозы и их влияние на функциональные требования.....	19
4.2. Преобразование технологических угроз в риски и оценка их вероятности.....	20
4.3. План обработки рисков и методы снижения.....	21
ЗАКЛЮЧЕНИЕ ПО РАЗДЕЛУ.....	23

1. ПОДГОТОВКА К ТЕСТИРОВАНИЮ ПРОДУКТА

1.1. Тестовый случай или тестовый сценарий

Тестовый случай или тестовый сценарий — это артефакт тестирования, который описывает совокупность условий, действий и ожидаемых результатов, необходимых для проверки отдельной функции программного продукта. Тестовый сценарий позволяет убедиться, что реализованный функционал работает корректно и соответствует требованиям технического задания.

В рамках тестирования программного продукта для планирования встреч были разработаны тестовые сценарии для проверки основных пользовательских функций: регистрации, авторизации, управления профилем, создания встреч, отправки приглашений, ответа на приглашения и просмотра расписания.

Стандартный тестовый сценарий включает следующие элементы:

1. Порядковый номер тестового сценария.
2. Название тестового сценария.
3. Проверяемая функция системы.
4. Предусловие для выполнения теста.
5. Последовательность действий пользователя.
6. Ожидаемый результат.
7. Результат выполнения теста.

Таблица 1 – Тестовый сценарий №1

Поле	Описание
Название	Регистрация нового пользователя
Функция	Регистрация пользователя
Действие	Создание новой учетной записи сотрудника в мобильном приложении
Ожидаемый результат	Пользователь успешно регистрируется, его данные сохраняются в базе данных, после чего он может выполнить вход в систему
Результат теста	Пройден
Предусловие	Приложение установлено и запущено. Серверная часть доступна. Пользователь ранее не был зарегистрирован в системе
Шаги теста	1. Открыть мобильное приложение. 2. Перейти на экран регистрации. 3. Ввести имя пользователя. 4. Ввести уникальный адрес электронной почты. 5. Ввести пароль. 6. Подтвердить регистрацию. 7. Проверить успешное создание учетной записи

Таблица 2 – Тестовый сценарий №2

Поле	Описание
Название	Авторизация пользователя
Функция	Вход в систему
Действие	Вход пользователя в приложение с использованием email и пароля
Ожидаемый результат	Пользователь успешно авторизуется, сервер возвращает JWT-токен, приложение переходит на главный экран
Результат теста	Пройден
Предусловие	Пользователь зарегистрирован в системе. Серверная часть доступна
Шаги теста	1. Открыть мобильное приложение. 2. Перейти на экран авторизации. 3. Ввести email зарегистрированного пользователя. 4. Ввести пароль. 5. Нажать кнопку входа. 6. Проверить переход на главный экран приложения

Таблица 3 – Тестовый сценарий №3

Поле	Описание
Название	Просмотр профиля пользователя
Функция	Профиль пользователя
Действие	Получение и отображение личных данных пользователя
Ожидаемый результат	На экране профиля отображаются актуальные данные пользователя: имя, email и дополнительные сведения профиля
Результат теста	Пройден
Предусловие	Пользователь авторизован в системе
Шаги теста	1. Выполнить вход в приложение. 2. Открыть раздел профиля. 3. Проверить отображение данных пользователя. 4. Убедиться, что данные соответствуют информации, сохраненной в системе

Таблица 4 – Тестовый сценарий №4

Поле	Описание
Название	Редактирование профиля пользователя
Функция	Изменение данных профиля
Действие	Пользователь изменяет данные своего профиля
Ожидаемый результат	Измененные данные сохраняются на сервере и отображаются в профиле после обновления
Результат теста	Пройден
Предусловие	Пользователь авторизован. Открыт экран профиля
Шаги теста	1. Открыть экран профиля. 2. Нажать кнопку редактирования. 3. Изменить имя пользователя или другие доступные данные. 4. Нажать кнопку сохранения. 5. Проверить, что данные обновились на экране. 6. Повторно открыть профиль и убедиться, что изменения сохранились

Таблица 5 – Тестовый сценарий №5

Поле	Описание
Название	Создание новой встречи
Функция	Создание встречи
Действие	Пользователь создает встречу, указывает дату, время, тему и участников
Ожидаемый результат	Встреча успешно создается, сохраняется в базе данных, выбранным участникам отправляются приглашения
Результат теста	Пройден
Предусловие	Пользователь авторизован. В системе существуют другие пользователи, которых можно пригласить
Шаги теста	1. Открыть главный экран приложения. 2. Перейти в раздел создания встречи. 3. Указать тему встречи. 4. Выбрать дату встречи. 5. Выбрать время начала и окончания встречи. 6. Указать место или ссылку на встречу. 7. Выбрать участников из списка пользователей. 8. Нажать кнопку создания встречи. 9. Проверить, что встреча создана

Таблица 6 – Тестовый сценарий №6

Поле	Описание
Название	Проверка ограничения времени встречи
Функция	Создание встречи по строгим часовым слотам
Действие	Пользователь пытается создать встречу с некорректным временем, например 09:10–10:10
Ожидаемый результат	Система не позволяет создать встречу с некорректным временем и выводит сообщение об ошибке
Результат теста	Пройден
Предусловие	Пользователь авторизован и находится на экране создания встречи
Шаги теста	1. Открыть форму создания встречи. 2. Ввести тему встречи. 3. Указать дату. 4. Попытаться выбрать время, не соответствующее часовому слоту. 5. Нажать кнопку создания встречи. 6. Проверить появление сообщения об ошибке

Таблица 7 – Тестовый сценарий №7

Поле	Описание
Название	Просмотр активных приглашений
Функция	Список приглашений
Действие	Пользователь открывает список полученных приглашений на встречи
Ожидаемый результат	В приложении отображается список активных приглашений с информацией о встрече и кнопками принятия или отклонения
Результат теста	Пройден
Предусловие	Пользователь авторизован. Для пользователя существуют активные приглашения
Шаги теста	1. Выполнить вход в приложение.2. Перейти в раздел приглашений.3. Проверить отображение списка приглашений.4. Убедиться, что у каждого приглашения есть информация о встрече.5. Проверить наличие кнопок «Принять» и «Отклонить»

Таблица 8 – Тестовый сценарий №8

Поле	Описание
Название	Принятие приглашения на встречу
Функция	Ответ на приглашение
Действие	Пользователь принимает приглашение на встречу
Ожидаемый результат	Статус приглашения изменяется на «принято», встреча появляется в личном расписании пользователя
Результат теста	Пройден
Предусловие	Пользователь авторизован. У пользователя есть активное приглашение на встречу
Шаги теста	1. Открыть раздел приглашений. 2. Выбрать активное приглашение. 3. Нажать кнопку «Принять». 4. Проверить изменение статуса приглашения. 5. Открыть расписание и убедиться, что встреча отображается

Таблица 9 – Тестовый сценарий №9

Поле	Описание
Название	Отклонение приглашения на встречу
Функция	Ответ на приглашение
Действие	Пользователь отклоняет приглашение на встречу
Ожидаемый результат	Статус приглашения изменяется на «отклонено», встреча не добавляется в личное расписание пользователя
Результат теста	Пройден
Предусловие	Пользователь авторизован. У пользователя есть активное приглашение на встречу
Шаги теста	1. Открыть раздел приглашений. 2. Выбрать активное приглашение. 3. Нажать кнопку «Отклонить». 4. Проверить изменение статуса приглашения. 5. Открыть расписание и убедиться, что отклоненная встреча не отображается

Таблица 10 – Тестовый сценарий №10

Поле	Описание
Название	Просмотр личного расписания
Функция	Расписание пользователя
Действие	Пользователь просматривает список подтвержденных встреч
Ожидаемый результат	В расписании отображаются только подтвержденные встречи пользователя за выбранный период
Результат теста	Пройден
Предусловие	Пользователь авторизован. У пользователя есть созданные или принятые встречи
Шаги теста	1. Выполнить вход в приложение. 2. Открыть раздел расписания. 3. Выбрать режим просмотра: день, неделя или месяц. 4. Проверить отображение встреч. 5. Убедиться, что отклоненные и неподтвержденные встречи не отображаются в расписании

1.2. Инструменты для проведения тестирования

Для тестирования программного продукта для планирования встреч использовались следующие инструменты:

- Android Studio — запуск и отладка мобильного приложения;
- Emulator Android / физическое Android-устройство — проверка работы клиентской части;
- IntelliJ IDEA — разработка и отладка серверной части;
- Spring Boot Test — тестирование серверных компонентов;
- Postman — проверка REST API-запросов;
- PostgreSQL — проверка хранения данных;
- Liquibase — проверка корректности миграций базы данных;
- GitHub — контроль версий и отслеживание изменений;
- Gradle — сборка проекта и запуск тестов;
- Logcat — анализ ошибок мобильного приложения;
- Swagger / OpenAPI — проверка доступных API-методов при наличии документации.

1.3. Тестирование (не-)функциональных требований

Check-list — это контрольный список проверок, которые необходимо выполнить в тестируемой системе. Он позволяет убедиться, что все основные функции программного продукта были протестированы и соответствуют требованиям технического задания.

В рамках тестирования приложения для планирования встреч был составлен чек-лист, включающий основные пользовательские сценарии: регистрацию, авторизацию, работу с профилем, создание встречи, обработку приглашений и просмотр расписания.

Таблица 11 – Чек-лист проведенного тестирования

№	Наименование тестового случая	Действия или входные данные	Ожидаемый результат	Фактический результат	Примечание
1	Регистрация нового пользователя	Ввести имя, email и пароль	Пользователь зарегистрирован	Пользователь зарегистрирован	Проверка уникальности email
2	Авторизация пользователя	Ввести email и пароль	Пользователь входит в систему	Вход выполнен успешно	JWT-токен получен
3	Ошибка при неверном пароле	Ввести неверный пароль	Отображается сообщение об ошибке	Ошибка отображается	Проверка валидации
4	Просмотр профиля	Открыть экран профиля	Данные пользователя отображаются	Данные отображаются корректно	Profile API
5	Редактирование профиля	Изменить имя пользователя	Данные сохраняются	Данные обновлены	Проверка сохранения на сервере
6	Создание встречи	Заполнить форму встречи	Встреча создается	Встреча создана	Meeting API
7	Проверка часового слота	Указать время 09:10–10:10	Создание запрещено	Ошибка отображается	Встречи только по целым часам
8	Приглашение участников	Выбрать участников встречи	Участникам отправлены приглашения	Приглашения созданы	Meeting participants
9	Просмотр приглашений	Открыть раздел приглашений	Список приглашений отображается	Приглашения отображаются	Invitation API
10	Принятие приглашения	Нажать «Принять»	Встреча появляется в расписании	Встреча добавлена	Статус accepted
11	Отклонение приглашения	Нажать «Отклонить»	Встреча не появляется в расписании	Встреча не добавлена	Статус declined
12	Просмотр расписания за день	Выбрать режим «День»	Отображаются встречи выбранного дня	Данные отображаются	Day view
13	Просмотр расписания за неделю	Выбрать режим «Неделя»	Отображаются встречи недели	Данные отображаются	Week view
14	Просмотр расписания за месяц	Выбрать режим «Месяц»	Отображаются встречи месяца	Данные отображаются	Month view
15	Защита API	Отправить запрос без токена	Доступ запрещен	Возвращается ошибка авторизации	JWT Security

Таблица 11.1 — Дополненная матрица требований

№	Тип требования	Суть требования	Критерий проверки	Компоненты архитектуры	Результат тестирования
1.1	Функциональное	Регистрация пользователя	Пользователь может создать учетную запись с уникальным email	Android-приложение, AuthController, UserService, БД	Пройден. Пользователь создается успешно
1.2	Функциональное	Авторизация пользователя	При корректных данных пользователь получает JWT-токен	Android-приложение, AuthController, JWT Security	Пройден. Авторизация работает корректно
1.3	Функциональное	Проверка уникальности email	Система не позволяет зарегистрировать двух пользователей с одним email	UserService, UserRepository, БД	Пройден. Дублирование email запрещено
1.4	Функциональное	Просмотр профиля	Пользователь видит данные своего профиля	Profile Screen, User API	Пройден. Данные отображаются
1.5	Функциональное	Редактирование профиля	Пользователь может изменить данные профиля	Android-приложение, UserService, БД	Пройден. Изменения сохраняются
1.6	Функциональное	Создание встречи	Пользователь может создать встречу с темой, датой, временем и участниками	Meeting Screen, MeetingController, MeetingService	Пройден. Встреча создается
1.7	Функциональное	Ограничение времени встречи	Встречи создаются только по строгим часовым слотам	MeetingService, Validation	Пройден. Некорректное время отклоняется
1.8	Функциональное	Приглашение участников	После создания встречи участникам создаются приглашения	MeetingService, ParticipantRepository	Пройден. Приглашения создаются
1.9	Функциональное	Просмотр активных	Пользователь видит список	Invitations Screen, Invitation	Пройден. Список

		приглашений	приглашений, ожидающих ответа	API	отображается
1.10	Функциональное	Принятие приглашения	При принятии встречи она появляется в расписании пользователя	InvitationService, Schedule API	Пройден. Встреча добавляется
1.11	Функциональное	Отклонение приглашения	При отклонении встреча не отображается в расписании	InvitationService, Schedule API	Пройден. Встреча не добавляется
1.12	Функциональное	Просмотр расписания	Пользователь видит подтвержденные встречи за выбранный период	Schedule Screen, Schedule API	Пройден. Расписание отображается
2.1	Нефункциональное	Производительность	Основные запросы API выполняются без заметной задержки	Spring Boot, PostgreSQL	Пройден. Критических задержек не выявлено
2.2	Нефункциональное	Надежность	Приложение не завершает работу аварийно при стандартных сценариях	Android-приложение, Backend	Пройден. Критических сбоев не выявлено
2.3	Нефункциональное	Безопасность	Закрытые API-методы недоступны без JWT-токена	Spring Security, JWT	Пройден. Защита работает
2.4	Нефункциональное	Сохраняемость данных	Данные пользователей, встреч и приглашений сохраняются в базе данных	PostgreSQL, Spring Data JPA	Пройден. Данные сохраняются корректно
2.5	Нефункциональное	Удобство интерфейса	Основные действия доступны через понятные экраны приложения	Android UI	Пройден. Интерфейс соответствует пользовательским сценариям
2.6	Нефункциональное	Совместимость	Приложение запускается на целевой	Android-приложение	Пройден. Приложение запускается

			версии Android		корректно
--	--	--	-------------------	--	-----------

2. ФОРМИРОВАНИЕ ПРОГРАММЫ И МЕТОДИКИ ИСПЫТАНИЙ СИСТЕМЫ

2.1. Программа и методика испытаний

1. Объект испытаний

Объектом испытаний является программный продукт для планирования встреч сотрудников внутри компании. Система состоит из мобильного Android-приложения и серверной части, реализованной в виде Spring Boot-приложения.

Мобильное приложение предоставляет пользователю интерфейс для регистрации, авторизации, управления профилем, создания встреч, просмотра приглашений и работы с личным расписанием. Серверная часть обеспечивает обработку REST API-запросов, хранение данных пользователей и встреч, проверку авторизации, работу с приглашениями и расписанием.

Назначение системы — автоматизация процесса планирования рабочих встреч между сотрудниками, упрощение приглашения участников и ведение персонального расписания.

2. Цель испытаний

Целью испытаний является проверка соответствия реализованного программного продукта требованиям технического задания, а также оценка качества и полноты реализации основных функций системы.

В процессе испытаний необходимо проверить:

- корректность регистрации и авторизации пользователей;
- правильность выдачи и проверки JWT-токена;
- возможность просмотра и редактирования профиля;
- корректность создания встреч;
- соблюдение ограничения на создание встреч только по часовым слотам;
- создание приглашений для выбранных участников;
- возможность принятия и отклонения приглашений;
- корректность отображения личного расписания;
- защиту серверных методов от неавторизованного доступа;

- устойчивость приложения при выполнении основных пользовательских сценариев.

3. Требования к программе

Во время испытаний проверяются следующие функции программного продукта:

- регистрация нового пользователя;
- авторизация пользователя по email и паролю;
- получение JWT-токена после успешного входа;
- защита API-запросов с использованием JWT;
- просмотр данных профиля пользователя;
- редактирование данных профиля;
- создание новой встречи;
- указание темы, даты, времени и места встречи;
- выбор участников встречи;
- создание приглашений для участников;
- просмотр списка активных приглашений;
- принятие приглашения на встречу;
- отклонение приглашения на встречу;
- отображение подтвержденных встреч в расписании;
- фильтрация расписания по периоду: день, неделя, месяц;
- проверка конфликтов времени при принятии встречи;
- сохранение данных в базе данных.

4. Требования к программной документации

В состав программной документации, предъявляемой на испытания, входят:

- техническое задание на разработку программного продукта;
- описание архитектуры системы;
- описание клиентской части Android-приложения;
- описание серверной части Spring Boot;
- описание базы данных и сущностей предметной области;
- описание REST API;
- диаграммы вариантов использования;
- диаграммы взаимодействия компонентов системы;
- руководство пользователя;
- чек-листы тестирования;
- тестовые сценарии;
- отчет о результатах испытаний.

5. Средства и порядок испытаний

Испытания проводятся с использованием следующих программных и технических средств:

- Android Studio для запуска и отладки мобильного приложения;
- Android Emulator или физическое Android-устройство для проверки клиентской части;
- IntelliJ IDEA для запуска и отладки серверной части;
- Spring Boot для реализации серверного приложения;
- Postman для проверки REST API-запросов;
- PostgreSQL Database для проверки хранения данных;
- Liquibase для проверки миграций базы данных;
- GitHub для контроля версий проекта;
- Gradle для сборки проекта и запуска тестов;
- Logcat для анализа ошибок Android-приложения.

Порядок испытаний включает следующие этапы:

1. Подготовка тестового окружения.
2. Запуск серверной части приложения.
3. Проверка подключения к базе данных.
4. Запуск мобильного приложения на эмуляторе или устройстве.
5. Выполнение тестовых сценариев.
6. Проверка корректности REST API-запросов через Postman.
7. Фиксация найденных ошибок.
8. Исправление ошибок разработчиками.
9. Повторное тестирование исправленных функций.
10. Оформление итоговых результатов испытаний.

6. Методы испытаний

Для проверки программного продукта применяются следующие методы испытаний:

- функциональное тестирование;
- регрессионное тестирование;
- тестирование пользовательского интерфейса;
- тестирование REST API;
- тестирование авторизации и прав доступа;
- тестирование валидации входных данных;
- тестирование работы с базой данных;
- тестирование устойчивости приложения;
- ручное тестирование пользовательских сценариев;
- модульное тестирование серверных компонентов.

	Проверяемы	Проверяемая	Метод	Ожидаемый
--	------------	-------------	-------	-----------

№	й модуль	функция	проверки	результат
1	Модуль регистрации	Создание нового пользователя	Ручное тестирование, проверка API	Пользователь создается в системе
2	Модуль авторизации	Вход по email и паролю	Ручное тестирование, Postman	Пользователь получает JWT-токен
3	Модуль безопасности	Доступ к защищенным методам	Отправка запроса без токена	Доступ запрещается
4	Модуль профиля	Просмотр профиля	Проверка экрана профиля и API	Данные пользователя отображаются
5	Модуль профиля	Редактирование профиля	Изменение данных пользователя	Новые данные сохраняются
6	Модуль встреч	Создание встречи	Заполнение формы встречи	Встреча создается в базе данных
7	Модуль встреч	Проверка часовых слотов	Ввод некорректного времени	Система выводит ошибку
8	Модуль приглашений	Создание приглашений	Создание встречи с участниками	Участникам создаются приглашения
9	Модуль приглашений	Просмотр активных приглашений	Открытие раздела приглашений	Список приглашений отображается
10	Модуль приглашений	Принятие приглашения	Нажатие кнопки «Принять»	Встреча появляется в расписании
11	Модуль приглашений	Отклонение приглашения	Нажатие кнопки «Отклонить»	Встреча не появляется в расписании
12	Модуль расписания	Просмотр расписания за день	Выбор режима «День»	Отображаются встречи выбранного дня
13	Модуль расписания	Просмотр расписания за неделю	Выбор режима «Неделя»	Отображаются встречи выбранной недели
14	Модуль	Просмотр	Выбор	Отображаются

	расписания	расписания за месяц	режима «Месяц»	я встречи выбранного месяца
15	База данных	Сохранение пользователей, встреч и приглашений	Проверка записей в БД	Данные сохраняются корректно

7. Заключение

Программа и методика испытаний составлены в соответствии с ГОСТ 19.301-79 и ГОСТ 34.603-92. Испытания охватывают основные функциональные возможности системы, включая регистрацию, авторизацию, управление профилем, создание встреч, обработку приглашений и просмотр расписания. В ходе испытаний проверяется работа как клиентской части Android-приложения, так и серверной части Spring Boot. Отдельное внимание уделяется корректности REST API, защите закрытых методов с помощью JWT, сохранению данных в базе данных и соблюдению бизнес-правил системы. Результаты испытаний подтверждают, что основные функции программного продукта реализованы корректно и соответствуют требованиям технического задания.

3. АНАЛИЗ И ОЦЕНКА РИСКОВ ПРИ РАЗРАБОТКЕ ПРОГРАММНОГО ПРОЕКТА

Таблица 12 – Описание рисков в формате «причина-риск-эффект»

№	Причина	Риск	Эффект
1	Недостаточная валидация входных данных при регистрации пользователя на стороне сервера.	Возможность создания дублирующих учётных записей с одинаковым email или регистрации с некорректными данными.	Нарушение целостности базы данных, невозможность корректной авторизации, сбои в работе UserService.
2	Отсутствие механизма обновления или отзыва JWT-токенов (refresh token / blacklist).	После компрометации токена злоумышленник сохраняет доступ к API до истечения срока его действия.	Несанкционированный доступ к данным пользователей и защищённым методам API, утечка персональных данных.
3	Отсутствие проверки конфликтов временных слотов на уровне БД (race condition при одновременных запросах).	Два пользователя могут одновременно создать встречи в одном и том же часовом слоте для одного участника.	Возникновение конфликтов в расписании, нарушение бизнес-логики системы, снижение доверия пользователей.
4	Отсутствие пагинации и ограничений при запросе расписания за большой временной период (например, год).	API-запрос расписания возвращает слишком большой объём данных, перегружая сервер и мобильный клиент.	Деградация производительности системы, зависание или краш Android-приложения, плохой UX.
5	Некорректная обработка ответов на приглашение: отсутствие проверки актуальности приглашения перед изменением статуса.	Пользователь может принять или отклонить уже устаревшее/удалённое приглашение, или принять дважды.	Некорректное состояние расписания, дублирование встреч, логические ошибки в InvitationService.
6	Отсутствие тестирования мобильного приложения на различных версиях Android и разрешениях экрана.	Адаптивная верстка корректно работает только на эмуляторе, но ломается на реальных устройствах.	Недоступность функционала для части пользователей, негативный пользовательский опыт, рост обращений в поддержку.

Таблица 12 – Описание рисков в формате «причина-риск-эффект» (продолжение)

7	Liquibase-миграции содержат ошибки или несовместимы с целевой версией PostgreSQL при первом развёртывании.	Схема базы данных не инициализируется корректно, приложение не запускается или работает некорректно.	Полный отказ системы при деплое, невозможность запуска серверной части, задержка ввода в эксплуатацию.
8	Отсутствие авторизационных проверок на уровне ресурсов: пользователь может получить или изменить чужие данные через API.	Злоумышленник передаёт чужой ID в запросе и получает доступ к профилю, встречам или приглашениям другого пользователя.	Утечка персональных данных, нарушение GDPR/законодательства о защите ПДн, репутационный ущерб.
9	Отсутствие обработки сетевых ошибок и таймаутов в Android-приложении при взаимодействии с сервером.	При потере соединения или медленном интернете приложение зависает без отображения сообщений об ошибке.	Негативный пользовательский опыт, потеря несохранённых данных (например, форма создания встречи), отток пользователей.
10	Отсутствие ограничений на количество создаваемых встреч и приглашений одним пользователем (rate limiting).	Пользователь или бот создаёт тысячи встреч/приглашений, перегружая БД и сервер Spring Boot.	Отказ в обслуживании (DoS) для остальных пользователей, деградация производительности или полный краш сервиса.
11	Недостаточное покрытие серверных компонентов юнит- и интеграционными тестами (Spring Boot Test).	Регрессионные баги в MeetingService, InvitationService или AuthController остаются незамеченными при разработке.	Выпуск продукта с критическими ошибками в бизнес-логике, необходимость экстренных патч-релизов.
12	Хранение паролей без надёжного хеширования (без bcrypt/Argon2) или передача данных без HTTPS.	При взломе БД или перехвате трафика злоумышленник получает учётные данные пользователей в открытом виде.	Массовая компрометация аккаунтов, потеря доверия пользователей, юридическая ответственность.

Таблица 13 – Определение важности рисков

№	«Причина-риск-эффект»	Вероятность (1-10)	Последствия (1-10)	Важность (В*П)
1	Возможность создания дублирующих учётных записей с одинаковым email или регистрации с некорректными данными.	7	8	56
2	После компрометации токена злоумышленник сохраняет доступ к API до истечения срока его действия.	6	9	54
3	Два пользователя могут одновременно создать встречи в одном и том же часовом слоте для одного участника.	6	8	48
4	API-запрос расписания возвращает слишком большой объём данных, перегружая сервер и мобильный клиент.	7	7	49
5	Пользователь может принять или отклонить уже устаревшее/удалённое приглашение, или принять дважды.	6	7	42
6	Адаптивная верстка корректно работает только на эмуляторе, но ломается на реальных устройствах.	8	6	48
7	Схема базы данных не инициализируется корректно, приложение не запускается или работает некорректно.	5	9	45
8	Злоумышленник передаёт чужой ID в запросе и получает доступ к профилю, встречам или приглашениям другого пользователя.	6	10	60
9	При потере соединения или медленном интернете приложение зависает без отображения сообщений об ошибке.	8	6	48
10	Пользователь или бот создаёт тысячи встреч/приглашений, перегружая БД и сервер Spring Boot.	5	8	40

Таблица 13 – Определение важности рисков (продолжение)

11	Регрессионные баги в MeetingService, InvitationService или AuthController остаются незамеченными при разработке.	7	8	56
12	При взломе БД или перехвате трафика злоумышленник получает учётные данные пользователей в открытом виде.	4	10	40

Таблица 14 – Матрица рисков

ВЕРОЯТНОСТЬ	Несущественные (1–2)	Низкие (3–4)	Средние (5–6)	Существенные (7–8)	Катастрофические (9–10)
Весьма вероятно (9–10)					
Вероятно (7–8)			6, 9	1, 4, 11	
Возможно (5–6)				3, 5, 10	2, 7, 8
Маловероятно (3–4)					12
Крайне маловероятно (1–2)					

Для понимания следующей таблицы, необходимо знать о стратегиях реагирования на риски проекта:

- Избегание (avoidance) – устранение источников риска. Например, если сотрудники собираются попасть в зону урагана, можно отменить их поездку, чтобы избежать риска;
- Передача (transference) – привлечение третьей стороны, принимающей риски на себя. Например, страховка – форма передачи риска, когда компания платит небольшую сумму третьей стороне, чтобы избежать полного финансового бремени, например, ущерба имуществу;
- Снижение (mitigation) – уменьшение вероятности воплощения угрозы. Например, промышленная пекарня не может избежать активных источников тепла, но может принять меры, чтобы снизить опасность для сотрудников и имущества;
- Принятие (acceptance) – пассивная форма предполагает осознанную готовность к угрозе, а активная форма – согласование плана действия при наступлении непредвиденных, но принимаемых обстоятельств. Например, если планируется мероприятие на открытом воздухе с 15%-ной вероятностью дождя, стратегия принятия такого небольшого риска может быть более разумной, чем изменение расписания.

Таблица 15 – Таблица планов реагирования на риски (продолжение)

№	Причина-риск-эффект (кратко)	Стратегия	Основной план	Отходной план
1	Недостаточная валидация входных данных при регистрации пользователя на стороне сервера.	Снижение (Mitigate)	Добавить серверную валидацию email через регулярные выражения и уникальный индекс на поле email в БД. Использовать аннотации @Valid в Spring Boot и возвращать HTTP 409 при дублировании.	Ввести мягкое ограничение: при регистрации с существующим email отправлять уведомление на этот адрес с предложением восстановить пароль.
2	Отсутствие механизма обновления или отзыва JWT-токенов (refresh token / blacklist).	Снижение (Mitigate)	Реализовать механизм refresh token: при истечении access-токена пользователь получает новый через refresh. При выходе из системы помещать токен в blacklist (Redis или таблица БД).	Установить минимальный TTL для JWT (например, 15 минут). При обнаружении подозрительной активности принудительно инвалидировать все сессии пользователя через смену секрета.
3	Отсутствие проверки конфликтов временных слотов на уровне БД (race condition при одновременных запросах).	Снижение (Mitigate)	Добавить уникальный индекс на (user_id, start_time) в таблице участников встреч. Использовать пессимистичную блокировку (SELECT FOR UPDATE) при создании встречи в MeetingService.	Реализовать оптимистичную блокировку через поле version в entity. При конфликте возвращать HTTP 409 и просить пользователя повторить попытку.
4	Отсутствие пагинации и ограничений при запросе расписания за большой временной период (например, год).	Снижение (Mitigate)	Добавить обязательные параметры date_from и date_to в API расписания. Ограничить максимальный диапазон 31 днём. Реализовать пагинацию с параметрами page и size (Spring Pageable).	Внедрить кэширование расписания через Spring Cache (Caffeine) с TTL 5 минут. При превышении лимита возвращать HTTP 400 с описанием допустимого диапазона.

Таблица 15 – Таблица планов реагирования на риски (продолжение)

5	Некорректная обработка ответов на приглашение: отсутствие проверки актуальности приглашения перед изменением статуса.	Снижение (Mitigate)	Добавить проверку статуса приглашения перед его обработкой: принимать ответ только если статус PENDING. Использовать оптимистичную блокировку на entity Invitation.	Реализовать идемпотентность: повторный ответ на уже обработанное приглашение возвращает текущий статус без ошибки и без изменений.
6	Отсутствие тестирования мобильного приложения на различных версиях Android и разрешениях экрана.	Снижение (Mitigate)	Проводить тестирование на эмуляторах с различными разрешениями (hdpi, xhdpi, xxhdpi) и на физических устройствах Android 8+. Использовать ConstraintLayout и адаптивные dp-единицы.	Ввести минимальные требования к устройству в описании приложения (Android 8.0+, экран от 4.7"). Настроить CI-прогон UI-тестов (Espresso) на виртуальных устройствах.
7	Liquibase-миграции содержат ошибки или несовместимы с целевой версией PostgreSQL при первом развёртывании.	Снижение (Mitigate)	Тестировать Liquibase-миграции в CI/CD pipeline на чистой PostgreSQL. Использовать тэги и контрольные суммы. Поддерживать скрипт отката (rollback) для каждой миграции.	Перед деплоем создавать резервную копию схемы БД. При сбое миграции автоматически откатывать на предыдущую версию и уведомлять DevOps-команду.
8	Отсутствие авторизационных проверок на уровне ресурсов: пользователь может получить или изменить чужие данные через API.	Снижение (Mitigate)	Внедрить проверку владельца ресурса на уровне сервисов (Resource-Based Authorization): сравнивать ID из JWT-токена с ID владельца запрашиваемого ресурса. Покрыть сценарий тестами.	Добавить интеграционные тесты безопасности (Spring Security Test) для всех защищённых эндпоинтов. Провести code review с фокусом на авторизацию перед релизом.

Таблица 15 – Таблица планов реагирования на риски (продолжение)

9	Отсутствие обработки сетевых ошибок и таймаутов в Android-приложении при взаимодействии с сервером.	Снижение (Mitigate)	Установить таймаут для всех HTTP-запросов в Android-клиенте (OkHttp: connect timeout 10s, read timeout 30s). Реализовать обработку исключений с отображением информативных сообщений об ошибке.	Добавить экран-заглушку с кнопкой 'Повторить' при сетевой ошибке. Кэшировать последнее состояние расписания локально (Room DB) для отображения в офлайн-режиме.
10	Отсутствие ограничений на количество создаваемых встреч и приглашений одним пользователем (rate limiting).	Снижение (Mitigate)	Внедрить rate limiting на уровне API-шлюза Spring Boot (Spring Cloud Gateway или Bucket4j): не более 20 создаваемых встреч в час на пользователя. Возвращать HTTP 429 при превышении.	Добавить CAPTCHA или подтверждение при массовом создании встреч. Ограничить максимальное количество активных встреч на одного организатора (например, 100).
11	Недостаточное покрытие серверных компонентов юнит- и интеграционными тестами (Spring Boot Test).	Снижение (Mitigate)	Установить минимальное покрытие тестами 70% для сервисного слоя (JaCoCo). Внедрить обязательное code review для MeetingService, InvitationService, AuthController. Запускать тесты в CI при каждом коммите.	Ввести регрессионное тестирование перед каждым релизом: ручной прогон чек-листа (Таблица 11) и автоматических тестов Postman-коллекции.
12	Хранение паролей без надёжного хеширования (без bcrypt/Argon2) или передача данных без HTTPS.	Снижение (Mitigate)	Хранить пароли только в виде хешей bcrypt (cost factor 12+) через Spring Security PasswordEncoder. Обеспечить HTTPS/TLS для всех API-запросов. Никогда не логировать пароли и токены.	Провести аудит безопасности кода перед первым деплоем. При обнаружении уязвимости — принудительный сброс паролей всех пользователей и уведомление по email.

4. АНАЛИЗ ЭКСПЛУАТАЦИИ ПРОЕКТА

4.1. Технологические угрозы и их влияние на функциональные требования

В соответствии с методикой анализа эксплуатационных рисков выделены три технологические угрозы, характерные для ИТ-инфраструктуры корпоративного приложения для планирования встреч. Влияние угроз оценено по 5-балльной шкале на ключевые функции системы, определённые матрицей требований (таблица 11). Шкала оценки приведена в примечании к таблице 16.

Таблица 16 – Оценка влияния технологических угроз на функциональные требования

№	Функциональное требование (из матрицы требований)	Угроза 1: Отказ дискового массива сервера БД	Угроза 2: DDoS-атака на публичный API	Угроза 3: Полный сбой облачного провайдера
1	Регистрация, авторизация и загрузка профиля	5 — чтение данных профиля невозможно, вход не выполняется	5 — API авторизации недоступен	5 — все серверы и службы остановлены
2	Создание новой встречи (выбор даты, времени, участников)	5 — запись новой встречи в БД не выполняется	5 — операции создания отклоняются сервером	5 — облачная инфраструктура недоступна
3	Бронирование локаций и управление временными слотами	5 — проверка доступности слотов невозможна	5 — запросы к сервису бронирования блокируются	5 — сервис управления локациями не отвечает
4	Просмотр расписания в оффлайн-режиме (кэш)	0 — полностью локальное чтение заэкшированных данных	0 — локальное расписание доступно	0 — отображаются ранее загруженные данные

№	Функциональное требование (из матрицы требований)	Угроза 1: Отказ дискового массива сервера БД	Угроза 2: DDoS-атака на публичный API	Угроза 3: Полный сбой облачного провайдера
5	Отправка и обработка приглашений (принятие/отклонение)	5 — статусы приглашений не сохраняются	5 — API обработки приглашений недоступен	5 — полная недоступность функционала
6	Оценка прошедшей встречи (рейтинг и комментариев)	5 — сохранение обратной связи (Feedback) невозможно	5 — отправка отзыва блокируется	5 — сервер не принимает запросы
7	Система push-уведомлений (напоминания)	2 — серверные триггеры не работают, но локальные напоминания срабатывают	2 — аналогично	2 — аналогично
8	Синхронизация расписания (календаря) с сервером	5 — актуализация календаря прерывается	5 — запросы на обновление отклоняются	5 — полная потеря связи с БД
9	Управление локальными настройками приложения	0 — изменения сохраняются в памяти устройства	0 — не зависит от API	0 — не зависит от серверов

Примечание — Шкала: 0 — не влияет; 1 — влияет незначительно; 2 — функциональность сохраняется, но снижается её быстродействие; 3 — функциональность доступна только в определённые интервалы времени; 4 — функциональность частично недоступна; 5 — функциональность недоступна полностью.

4.2. Преобразование технологических угроз в риски и оценка их вероятности

На основе формулы «Риск = Угроза + Уязвимость + Актив» выполнено преобразование угроз в конкретные риски. Для каждого риска определена вероятность возникновения при эксплуатации корпоративного приложения (по шкале от 1 до 10, используемой в разделе 3).

Таблица 17 — Реестр рисков эксплуатации с оценкой вероятности

Угроза	Уязвимость (особенность актива)	Актив	Сформулированный риск (угроза + уязвимость)	Вероятность (1–10)
Отказ дискового массива сервера БД	Отсутствие синхронной реплики данных в реальном времени и резервного ЦОД	База данных пользователей, встреч, локаций и приглашений	Полная потеря или длительная недоступность корпоративного расписания из-за разрушения RAID-массива основного сервера БД	4
DDoS-атака на публичный API приложения	Публичная доступность API (REST) без внешней системы фильтрации трафика	Серверная часть, веб-сервисы авторизации и обработки встреч	Отказ в обслуживании для всех сотрудников вследствие перегрузки API-шлюза вредоносным трафиком	7
Полный сбой облачного провайдера	Размещение всех компонентов (БД, серверного API) в одном регионе одного облачного провайдера	Виртуальные машины с серверной частью приложения и хранилищем БД	Длительная недоступность корпоративного календаря и невозможность планирования встреч из-за аварии на стороне единственного облачного провайдера	3

4.3. План обработки рисков и методы снижения

В соответствии с подходом Disaster Recovery для каждого риска выбрана стратегия реагирования (снижение, передача, принятие), разработаны основные и резервные мероприятия, обоснованные конкретными изменениями в архитектуре серверной части и процессах эксплуатации.

Таблица 18 — Планы реагирования на риски эксплуатации

Риск (кратко)	Стратегия	Основной план (снижение / обработка)	Отходной план и обоснование
Потеря данных календаря из-за отказа дискового массива БД	Снижение	Внедрить георезервированную реплику БД (Master-Slave) в другом ЦОД с задержкой не более 1 минуты (RPO=1 мин). Настроить ежечасное полное резервное копирование на удалённое S3-совместимое хранилище. Использовать RAID 10 на основном сервере БД.	При недоступности основной БД автоматически переключиться на реплику. Время восстановления RTO не превысит 15 минут. Все операции создания встреч и приглашений будут перенаправлены на резервный узел без потери данных.
Отказ в обслуживании из-за DDoS на API сервера	Снижение	Подключить внешний сервис защиты от DDoS-атак с фильтрацией на уровне L3/L4 и L7. Настроить rate limiting на API-шлюзе, кэширование статических ответов. Постоянно мониторить аномалии трафика.	В случае массовой атаки приложение переходит в оффлайн-режим. Сотрудники смогут просматривать своё закешированное расписание на текущую неделю. Локальные напоминания о встречах продолжают работать в штатном режиме.
Длительная недоступность из-за сбоя облачного провайдера	Снижение + Передача	Реализовать Multi-Site Solution: развернуть дублирующую инфраструктуру в другом облаке с предварительно настроенными образами виртуальных машин (backend-серверов) и автоматической репликацией данных БД. Заключить SLA с	При аварии основного облака запустить резервную инфраструктуру в течение 30 минут (RTO=30 мин). Обоснование: параллельная инфраструктура позволяет восстановить сервисы планирования встреч без

Риск (кратко)	Стратегия	Основной план (снижение / обработка)	Отходной план и обоснование
		провайдером с уровнем доступности не ниже 99,95%.	существенной потери времени, что критично для координации бизнес-процессов компании.

ЗАКЛЮЧЕНИЕ ПО РАЗДЕЛУ

Проведён анализ эксплуатационной устойчивости клиент-серверной системы планирования встреч. Определены критические точки отказа ИТ-инфраструктуры, влияющие на доступность серверных и облачных функций (авторизация пользователей, создание встреч, бронирование локаций и синхронизация календарей через REST API).

Для каждой технологической угрозы рассчитано её влияние на ключевые сценарии использования корпоративного приложения, оценена вероятность реализации рисков и предложены конкретные архитектурные и организационные меры в рамках стратегии Disaster Recovery (включая георезервирование базы данных PostgreSQL и переход мобильного Android-клиента в автономный режим работы с кэшированными данными при сбоях сети).

Внедрение описанных решений позволит обеспечить целевые показатели RPO ≤ 1 минуты и RTO ≤ 30 минут для критически важных внутренних бизнес-процессов компании, что гарантирует высокий уровень доступности сервиса и стабильную координацию рабочих графиков сотрудников.